QUESTION-13:

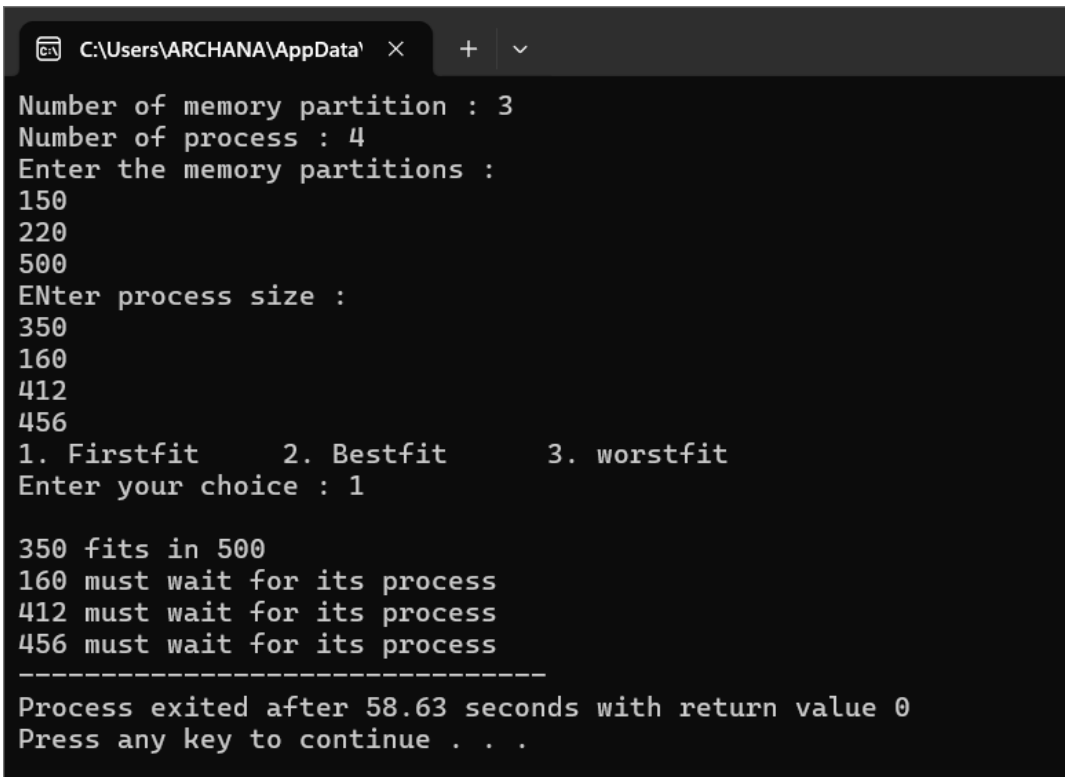
Construct a C program to implement various memory allocation strategies.

PROGRAM :

```
#include<stdio.h>
void bestfit(int mp[],int p[],int m,int n){ int j=0;
for(int i=0;i<n;i++)
{ if(mp[i]>p[j])
{ printf("\n%d fits in %d",p[j],mp[i]); mp[i]=mp[i]-p[j++]; i=i-1;
}
}
for(int i=j;i<m;i++)
{
printf("\n%d must wait for its process",p[i]); }
}
void rsort(int a[],int n)
{ for(int i=0;i<n;i++)
{ for(int j=0;j<n;j++)
{ if(a[i]>a[j])
{ int t=a[i]; a[i]=a[j]; a[j]=t;
}
}
}
}
void sort(int a[],int n){ for(int i=0;i<n;i++){ for(int j=0;j<n;j++){ if(a[i]<a[j]){ int t=a[i]; a[i]=a[j]; a[j]=t;
}
}
} }
void firstfit(int mp[],int p[],int m,int n){ sort(mp,n); sort(p,m); bestfit(mp,p,m,n);
}
void worstfit(int mp[],int p[],int m,int n){ rsort(mp,n); sort(p,m); bestfit(mp,p,m,n);
```

```
} int main(){ int m,n,mp[20],p[20],ch;
printf("Number of memory partition : ");
scanf("%d",&n); printf("Number of process : ");
scanf("%d",&m); printf("Enter the memory partitions : \n");
for(int i=0;i<n;i++){ scanf("%d",&mp[i]);
}
printf("ENter process size : \n");
for(int i=0;i<m;i++){ scanf("%d",&p[i]);
}
printf("1. Firstfit\t2. Bestfit\t3. worstfit\nEnter your choice : ");
scanf("%d",&ch);
switch(ch){ case 1: bestfit(mp,p,m,n); break;
case 2: firstfit(mp,p,m,n); break;
case 3: worstfit(mp,p,m,n); break;
default: printf("invalid"); break;
}
}
```

OUTPUT:



QUESTION-14:

Construct a C program to organize the file using single level directory.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#define BUFFER_SIZE 4096
// Copy function
void copy() {
   const char *source_file =
"C:/Users/ARCHANA/OneDrive/Pictures/Documents/Desktop/input.txt";
   const char *destination_file =
"C:/Users/ARCHANA/OneDrive/Pictures/Documents/Desktop/output.txt";


   int source_fd = open(source_file, O_RDONLY);
   if (source_fd < 0) {
      perror("Error opening source file");
      return;
   }
   int dest_fd = open(destination_file, O_WRONLY | O_CREAT | O_TRUNC, 0666);
   if (dest_fd < 0) {
      perror("Error opening destination file");
      close(source_fd);
      return;
   }
   char buffer[BUFFER_SIZE];
   ssize_t bytesRead, bytesWritten;
   while ((bytesRead = read(source_fd, buffer, BUFFER_SIZE)) > 0) {
      bytesWritten = write(dest_fd, buffer, bytesRead);
      if (bytesWritten != bytesRead) {
```

```
            perror("Error writing to file");
            break;
        }
    }
    close(source_fd);
    close(dest_fd);

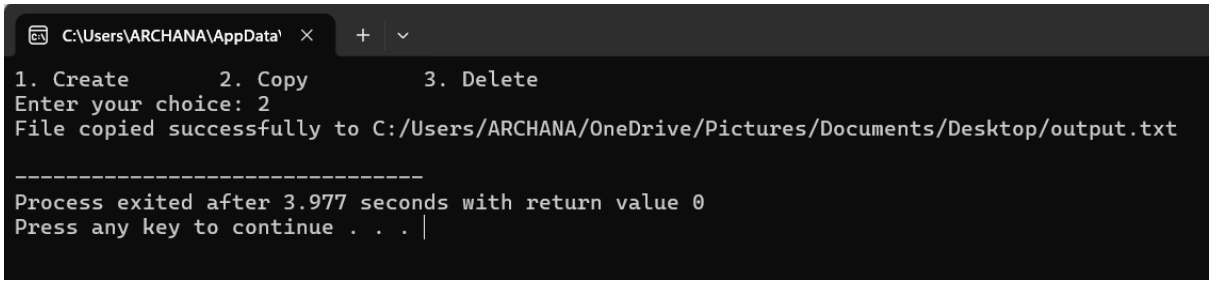
    printf("File copied successfully to %s\n", destination_file);

// Create function
void create() {
    const char *file_path = "C:\\Users\\Sahasra\\OneDrive\\Desktop\\New Text Document.txt";
    FILE *fp = fopen(file_path, "w");
    if (fp == NULL) {
        perror("Error creating file");
        return;
    }
    fprintf(fp, "This is a sample text file created by C program.\n");
    fclose(fp);
    printf("File created successfully at %s\n", file_path);

// Delete function
void delete_file() {
    const char *file_path = "C:\\Users\\Sahasra\\OneDrive\\Desktop\\New Text Document.txt";
    if (remove(file_path) == 0)
        printf("File deleted successfully.\n");
    else
        perror("Error deleting file");
}
```

```
int main() {
    int n;
    printf("1. Create \t2. Copy \t3. Delete\nEnter your choice: ");
    scanf("%d", &n);
    switch (n) {
        case 1:
            create();
            break;
        case 2:
            copy();
            break;
        case 3:
            delete_file();
            break;
        default:
            printf("Invalid choice.\n");
    }
    return 0;
}
```

OUTPUT:



QUESTION-15:

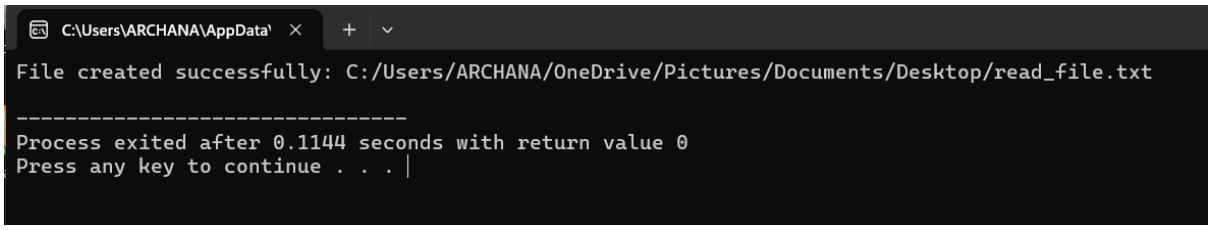
Design a C program to organize the file using two level directory structure.

PROGRAM:

```
#include <stdio.h
```

```
#include <stdlib.h>
int main() {
    // Path to your Desktop file
    char filePath[] =
"C:/Users/ARCHANA/OneDrive/Pictures/Documents/Desktop/read_file.txt";
    // Create (or overwrite if it exists)
    FILE *file = fopen(filePath, "w");
    if (file == NULL) {
        printf("Error creating file.\n");
        return 1;
    }
    // Write some content
    fprintf(file, "This is an example file content.")
    fclose(file);
    printf("File created successfully: %s\n", filePath);
    return 0;
}
```

OUTPUT:



QUESTION-16:

Develop a C program for implementing random access file for processing the employee details.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct Employee {
    int empId;
```

```
    char empName[50];
    float empSalary;
};
int main() {
    FILE *filePtr;
    struct Employee emp;
    // Open file for read/write, create if it doesn't exist
    filePtr = fopen("employee.dat", "rb+");
    if (filePtr == NULL) {
        filePtr = fopen("employee.dat", "wb+");
        if (filePtr == NULL) {
            printf("Error creating the file.\n");
            return 1;
        }
    }
    int choice;
    do {
        printf("\n--- Employee Database Menu ---\n");
        printf("1. Add Employee\n");
        printf("2. Display Employee Details\n");
        printf("3. Update Employee Details\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice)
        switch (choice) {
        case 1:
            printf("Enter Employee ID: ");
            scanf("%d", &emp.empId);
            printf("Enter Employee Name: ");
            scanf("%s", emp.empName);
```

```
    printf("Enter Employee Salary: ");
    scanf("%f", &emp.empSalary);
    fseek(filePtr, (emp.empId - 1) * sizeof(struct Employee), SEEK_SET);
    fwrite(&emp, sizeof(struct Employee), 1, filePtr);
    printf("Employee details added successfully.\n");
    break;
case 2:
    printf("Enter Employee ID to display: ");
    scanf("%d", &emp.empId)
    fseek(filePtr, (emp.empId - 1) * sizeof(struct Employee), SEEK_SET);
    if (fread(&emp, sizeof(struct Employee), 1, filePtr)) {
        printf("\n--- Employee Details ---\n");
        printf("Employee ID   : %d\n", emp.empId);
        printf("Employee Name : %s\n", emp.empName);
        printf("Employee Salary: %.2f\n", emp.empSalary);
    } else {
        printf("Employee not found.\n");
    }
    break;
case 3:
    printf("Enter Employee ID to update: ");
    scanf("%d", &emp.empId);
    fseek(filePtr, (emp.empId - 1) * sizeof(struct Employee), SEEK_SET);
    if (fread(&emp, sizeof(struct Employee), 1, filePtr)) {
        printf("Enter New Employee Name: ");
        scanf("%s", emp.empName)
        printf("Enter New Employee Salary: ");
        scanf("%f", &emp.empSalary);
```

```
            fseek(filePtr, (emp.empId - 1) * sizeof(struct Employee), SEEK_SET);
            fwrite(&emp, sizeof(struct Employee), 1, filePtr);
            printf("Employee details updated successfully.\n");
        } else {
            printf("Employee not found.\n");
        }
        break;
    case 4:
        printf("Exiting Employee Database.\n");
        break;

    default:
        printf("Invalid choice. Please try again.\n");
    }
  } while (choice != 4);
  fclose(filePtr);
  return 0;
}
```

OUTPUT:

